

TECHNICAL IMPLEMENTATION GUIDE

DayFlow Backend

NestJS + MongoDB Architecture, Authentication, and Migration from Firebase

Scope: Backend (NestJS/MongoDB) and its integration with the Next.js frontend

Purpose: Study reference for viva / interview preparation

Based on: Direct inspection of the source code in this repository

Table of Contents

1. Overall Architecture
2. Firebase → NestJS + MongoDB Migration
3. Backend Folder Structure
4. NestJS Modules and Their Responsibilities
5. Controllers, Services, DTOs and Schemas
6. MongoDB / Mongoose Implementation
7. JWT Authentication
8. Access Token + Refresh Token Flow
9. Password Hashing
10. Forgot Password, Password Reset, and Resend
11. User Authentication and Ownership / Authorization
12. All API Endpoints, Grouped by Module
13. Frontend → api-client → NestJS → MongoDB Request Flow
14. Validation and Error Handling
15. Notifications, Push, and Background Jobs
16. Important Security Decisions
17. Important Bugs Discovered and Fixed During Migration
18. Environment Variables
19. How to Run Frontend and Backend Locally
20. End-to-End Example: Creating a Task
21. What Was Removed From Firebase, and Why
22. Important Limitations That Still Exist

1. Overall Architecture

WHAT

DayFlow is a personal life-management app split into two independently-run projects that live in the same Git repository:

- **src/** — a Next.js 16 (App Router) frontend, running on port `9002`.
- **backend/** — a standalone NestJS API server, running on port `3001`, talking to MongoDB Atlas through Mongoose.

They are two separate Node.js processes with two separate `package.json` files and two separate `tsconfig.json` files. The frontend never talks to MongoDB directly — every piece of data goes through the NestJS backend's REST API.

```
Browser
|
▼
Next.js frontend (src/app/**)      - React components, pages
|  calls functions in
▼
src/lib/api-client.ts             - fetch() wrapper, attaches JWT
|  HTTP requests (JSON) to http://localhost:3001/api/*
▼
NestJS backend (backend/src/**)   - Express under the hood
|  Controllers → Services → Mongoose Models
▼
MongoDB Atlas (cluster: dayflow database)
```

WHY THIS SHAPE

The project originally used Firebase (Firebase Auth + Firestore + Firebase Storage + Firebase Admin) directly from the Next.js app, including a handful of Next.js API routes that used `firebase-admin` on the server side. Over the course of this project the entire data and auth layer was migrated to a self-hosted NestJS + MongoDB backend, so the app is no longer dependent on any Firebase product.

Why NestJS specifically

NestJS is a Node.js framework built on top of Express that enforces a consistent, modular structure (Controllers, Services, Modules, DTOs, Guards). This repository leans on that structure heavily: every feature (tasks, expenses, habits, ...) is implemented as its own NestJS module with the same shape, which is what makes the codebase predictable once you understand one module.

Authentication model at a glance

DayFlow uses stateless **JWT access tokens** for authorizing API calls, plus a stateful **refresh token** stored as an httpOnly cookie and tracked in MongoDB, so a session can be revoked (logout, password reset) even though JWTs themselves cannot be "un-signed". This is explained fully in Sections 7–9.

2. Firebase → NestJS + MongoDB Migration

WHAT CHANGED

Every Firebase product used by the app was replaced by an equivalent built on the new backend, one feature at a time.

Firestore product	Used for	Replaced by
Firebase Auth (email/password)	Sign in / sign up	NestJS <code>AuthModule</code> issuing JWTs, passwords hashed with bcrypt (Section 7–9)
Firebase Auth (anonymous)	"Continue as Guest"	<code>POST /api/auth/guest</code> — a real MongoDB user document with <code>isAnonymous: true</code> and no email/password
Firebase Auth (<code>sendPasswordResetEmail</code>)	Forgot password	<code>POST /api/auth/forgot-password / reset-password</code> + Resend (Section 10)
Cloud Firestore	Per-user documents under <code>users/{uid}/data/{collection}</code>	Dedicated MongoDB collections, one per feature, each document owned by a real <code>userId</code> field (Section 6)
Firebase Storage	Profile picture upload	Image is read client-side as a base64 <code>data:</code> URI and stored directly in the <code>photoUrl</code> field of the user document
Firebase Admin SDK (in Next.js API routes)	Server-side Firestore reads/writes for cron jobs (<code>api/cron/send-reminders</code> , <code>api/cron/water-check</code>) and verifying ID tokens for push routes (<code>api/push/*</code>)	Logic ported into the NestJS <code>JobsModule</code> , querying MongoDB directly (Section 15)

Migration strategy

The migration was done incrementally and verified at each step, not as one big rewrite:

1. **Auth first** — because every other feature depends on having an authenticated user id available.
2. **One feature at a time** — Notes, Credentials, Notifications/Push/Settings, Expenses, Habits, Planner, then the Dashboard widgets that reuse those same features, then Profile.
3. **Verification by curl** — every new/changed endpoint was exercised directly with `curl` against the running backend (register a throwaway user, call the endpoint, inspect the JSON, delete the test data) before wiring the frontend to it.
4. **Full Firebase removal last** — once every feature had a working backend equivalent, all Firebase SDK code, config files, and npm packages were deleted in one pass (Section 21).

WHERE

The single biggest structural difference: Firestore's old shape stored *one document per collection per user*, e.g. `users/{uid}/data/todos` holding `{ items: [...] }`. MongoDB instead stores *one document per item* (e.g. one document per task) inside a shared `tasks` collection, with every document carrying its own `userId` field. This is why almost every list in the app now uses a real MongoDB `_id` as its identifier instead of a client-generated string id.

3. Backend Folder Structure

```
backend/
├── src/
│   ├── main.ts                - application bootstrap (Section 7 & 14)
│   ├── app.module.ts          - root module, registers Mongoose connection + every feature module
│   ├── common/
│   │   ├── decorators/
│   │   │   └── current-user.decorator.ts
│   │   ├── guards/
│   │   │   └── jwt-auth.guard.ts
│   │   └── filters/
│   │       └── http-exception.filter.ts
│   └── modules/
│       ├── auth/
│       │   ├── auth.controller.ts
│       │   ├── auth.service.ts
│       │   ├── auth.module.ts
│       │   ├── mail.service.ts    - Resend wrapper
│       │   ├── dto/auth.dto.ts
│       │   ├── schemas/refresh-session.schema.ts
│       │   ├── schemas/password-reset-token.schema.ts
│       │   └── strategies/jwt.strategy.ts
│       ├── users/
│       │   ├── users.controller.ts
│       │   ├── users.service.ts
│       │   ├── users.module.ts
│       │   ├── dto/update-user.dto.ts
│       │   └── schemas/user.schema.ts
│       ├── tasks/tasks.module.ts    - schema + DTO + service + controller in ONE file
│       ├── expenses/expenses.module.ts - same pattern (Transaction + Budget)
│       ├── habits/habits.module.ts  - same pattern (Habit + GymSettings)
│       ├── planner/planner.module.ts - same pattern (PlannerSchedule)
│       ├── notes/notes.module.ts    - same pattern (Note)
│       ├── credentials/credentials.module.ts - same pattern (Credential)
│       ├── notifications/notifications.module.ts - same pattern (Notification)
│       ├── push/push.module.ts      - same pattern (PushSubscription)
│       ├── jobs/jobs.module.ts      - cron-style endpoints, reuses schemas above
│       └── health/health.module.ts  - DB ping endpoint
├── .env / .env.example
├── nest-cli.json
├── package.json
└── tsconfig.json
```

WHY TWO DIFFERENT FILE STYLES

`auth` and `users` are split across multiple files (controller / service / dto / schema each in their own file) because they carry more logic and are imported by other modules. Every other feature module (`tasks` , `expenses` , `habits` , `planner` , `notes` , `credentials` , `notifications` , `push`) instead packs its schema, DTO, service, controller, and module registration into a *single dense file*, because each one is a small, self-contained CRUD feature with no cross-module dependencies. Once you understand `tasks.module.ts` (Section 5), you can read any of the others the same way.

4. NestJS Modules and Their Responsibilities

Module	File	MongoDB collection(s)	Responsibility
AuthModule	modules/auth/auth.module.ts	refresh_sessions , password_reset_tokens	Register, login, guest login, refresh, logout, change email/password, forgot/reset password
UsersModule	modules/users/users.module.ts	users	Read/update the current user's profile (username, email, phone, photo, theme)
TasksModule	modules/tasks/tasks.module.ts	tasks	To-do items shown on the Dashboard
ExpensesModule	modules/expenses/expenses.module.ts	transactions , budgets	Income/expense transactions and the monthly budget
HabitsModule	modules/habits/habits.module.ts	habits , gym_settings	Habit tracker + the consolidated "Gym Tracker" data blob
PlannerModule	modules/planner/planner.module.ts	planner_schedules	The weekly schedule (one document per user, a map of day → items)
NotesModule	modules/notes/notes.module.ts	notes	Text and checklist notes
CredentialsModule	modules/credentials/credentials.module.ts	credentials	The password manager's stored entries
NotificationsModule	modules/notifications/notifications.module.ts	notifications	In-app reminder notifications
PushModule	modules/push/push.module.ts	push_subscriptions	Browser Web Push subscription storage
JobsModule	modules/jobs/jobs.module.ts	(reads Notification/Habit/PushSubscription)	Cron-triggered reminder + hydration push notifications
HealthModule	modules/health/health.module.ts	—	Database connectivity check, GET /api/health/database

All twelve modules are registered once, in `backend/src/app.module.ts`, alongside the global Mongoose connection setup.

5. Controllers, Services, DTOs and Schemas

Every feature module follows the exact same four-part pattern. Here is the complete, real implementation of the Tasks feature (`backend/src/modules/tasks/tasks.module.ts`), annotated part by part:

1. Schema — what gets stored in MongoDB

```
@Schema({ timestamps: true, collection: 'tasks' })
class Task {
  @Prop({ type: Types.ObjectId, ref: 'User', required: true, index: true })
  userId!: Types.ObjectId;

  @Prop({ required: true, trim: true, maxlength: 500 })
  text!: string;

  @Prop({ default: false })
  completed!: boolean;

  @Prop({ enum: ['high', 'medium', 'low'] })
  priority?: string;
}
const TaskSchema = SchemaFactory.createForClass(Task);
```

WHAT

A Mongoose schema, declared with NestJS's decorator syntax (`@Schema` / `@Prop`) instead of a plain Mongoose `new Schema({...})` call. It describes the shape, types, and constraints of a document in the `tasks` collection.

2. DTO — what a request body must look like

```
class TaskDto {
  @IsString() @MaxLength(500) text!: string;
  @IsOptional() @IsBoolean() completed?: boolean;
  @IsOptional() @IsIn(['high', 'medium', 'low']) priority?: string;
}
```

WHAT

A plain class decorated with `class-validator` decorators. NestJS's global `ValidationPipe` (Section 14) uses this class to validate every incoming request body before it reaches the controller method.

3. Service — the actual database logic

```

@Injectable()
class TasksService {
  constructor(@InjectModel(Task.name) private readonly model: Model<Task>) {}

  findAll(userId: string) {
    return this.model.find({ userId }).sort({ createdAt: -1 }).lean().exec();
  }
  create(userId: string, dto: TaskDto) {
    return this.model.create({ ...dto, userId });
  }
  update(userId: string, id: string, dto: TaskDto) {
    return this.model.findOneAndUpdate({ _id: id, userId }, dto, { new: true, runValidators: true }).exec();
  }
  remove(userId: string, id: string) {
    return this.model.deleteOne({ _id: id, userId }).exec();
  }
}

```

WHY EVERY QUERY INCLUDES `USERID`

This is the ownership/authorization mechanism for the whole app (see Section 11). A user can never read, update, or delete another user's task, because the query itself only ever matches documents whose `userId` equals the id taken from their own verified JWT.

4. Controller — the HTTP routes

```

@Controller('tasks')
@UseGuards(JwtAuthGuard)
class TasksController {
  constructor(private readonly service: TasksService) {}

  @Get() all(@CurrentUser() user: { sub: string }) { return this.service.findAll(user.sub); }
  @Post() create(@CurrentUser() user, @Body() dto: TaskDto) { return this.service.create(user.sub, dto); }
  @Patch(':id') update(@CurrentUser() user, @Param('id') id: string, @Body() dto: TaskDto) { return this.service.update(user.sub, id, dto); }
  @Delete(':id') remove(@CurrentUser() user, @Param('id') id: string) { return this.service.remove(user.sub, id); }
}

```

WHERE

`@UseGuards(JwtAuthGuard)` on the whole controller means every route in it requires a valid JWT (Section 7). `@CurrentUser()` is a custom parameter decorator ([common/decorators/current-user.decorator.ts](#)) that pulls the decoded JWT payload off the request object — it is never something the client can set directly.

Module registration

```

@Module({
  imports: [MongooseModule.forFeature([{ name: Task.name, schema: TaskSchema }])],
  controllers: [TasksController],
  providers: [TasksService],
})
export class TasksModule {}

```

Every other feature module (Expenses, Habits, Planner, Notes, Credentials, Notifications, Push) is built from the exact same four pieces, just with a different schema/DTO shape. The only structural variations worth knowing:

- **Expenses** and **Habits** each declare *two* schemas in one file (Transaction+Budget, Habit+GymSettings) because those two pieces of data are logically related.

- **Habits'** `PATCH /habits` and **Planner's** `PUT /planner` are *replace-all* operations, not per-item updates — see Section 17 for why this matters.

6. MongoDB / Mongoose Implementation

Connecting to MongoDB

WHERE

backend/src/app.module.ts

```
MongooseModule.forRootAsync({
  imports: [ConfigModule],
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    uri: config.getOrThrow<string>('MONGODB_URI'),
    serverSelectionTimeoutMS: Number(config.get('MONGODB_SERVER_SELECTION_TIMEOUT_MS', '5000')),
    connectTimeoutMS: Number(config.get('MONGODB_CONNECT_TIMEOUT_MS', '10000')),
  }),
});
```

This registers a single, shared MongoDB connection (to a MongoDB Atlas cluster) for the whole application. Every feature module then registers its own schema against that shared connection with `MongooseModule.forFeature([...])`.

Key Mongoose concepts used throughout the backend

Concept	What it does	Where used
<code>@Schema({ timestamps: true })</code>	Automatically adds and maintains <code>createdAt</code> / <code>updatedAt</code> fields on every document	Every schema in the backend
<code>@Prop({ type: Types.ObjectId, ref: 'User' })</code>	Declares a field as a reference to a document in the <code>users</code> collection (the ownership foreign key)	Every feature schema's <code>userId</code> field
<code>index: true</code> / <code>schema.index({...})</code>	Creates a MongoDB index for faster queries (e.g. all of a user's transactions sorted by date)	<code>tasks</code> , <code>transactions</code> , <code>notes</code> , <code>notifications</code> , <code>credentials</code> , <code>push_subscriptions</code>
<code>expires: 0</code> on a Date field	A TTL index — MongoDB automatically deletes the document once that date/time has passed	<code>RefreshSession.expiresAt</code> , <code>PasswordResetToken.expiresAt</code>
<code>.lean()</code>	Returns plain JavaScript objects instead of full Mongoose Documents (faster, since the app only reads the data and doesn't need Mongoose's change-tracking)	Most read-only queries (<code>find</code> , list endpoints)
<code>findOneAndUpdate(..., { upsert: true })</code>	"Create if it doesn't exist, otherwise update it" — used for one-document-per-user settings	<code>Budget</code> , <code>GymSettings</code> , <code>PlannerSchedule</code>
<code>select: false</code> on a Prop	Excludes that field from query results by default (it has to be explicitly requested)	<code>User.passwordHash</code> — see Section 9
<code>select('+passwordHash')</code>	Explicitly opts back in to a <code>select:false</code> field for that one query	<code>UserService.findByEmail</code> , <code>findByIdWithPassword</code>

MongoDB collections in this project

```
users, refresh_sessions, password_reset_tokens,  
tasks, transactions, budgets,  
habits, gym_settings,  
planner_schedules,  
notes, credentials, notifications, push_subscriptions
```

7. JWT Authentication

WHAT

JWT (JSON Web Token) is a signed, self-contained string that encodes a small payload (here: `{ sub: userId, email }`) plus an expiry time. Because it is cryptographically signed with a server-only secret, the server can trust its contents without needing to look anything up — that is what "stateless" authentication means.

How the pieces fit together

Piece	File	Role
<code>JwtModule.registerAsync(...)</code>	<code>modules/auth/auth.module.ts</code>	Registers <code>@nestjs/jwt</code> 's <code>JwtService</code> (used to sign/verify tokens). Registered with empty default options because each individual sign call supplies its own secret and expiry.
<code>JwtStrategy</code>	<code>modules/auth/strategies/jwt.strategy.ts</code>	A Passport.js strategy that extracts the token from the <code>Authorization: Bearer <token></code> header and verifies its signature against <code>JWT_SECRET</code> .
<code>JwtAuthGuard</code>	<code>common/guards/jwt-auth.guard.ts</code>	A one-line NestJS Guard (<code>extends AuthGuard('jwt')</code>) applied to controllers with <code>@UseGuards(JwtAuthGuard)</code> to require a valid token.
<code>CurrentUser</code> decorator	<code>common/decorators/current-user.decorator.ts</code>	Pulls the verified payload (<code>request.user</code> , set by Passport after the guard succeeds) into a controller method's parameter.

The actual JwtStrategy code

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: config.getOrThrow<string>('JWT_SECRET'),
    });
  }
  validate(payload: { sub: string; email?: string }) {
    return payload; // becomes request.user
  }
}
```

HOW AUTHORIZATION ACTUALLY HAPPENS ON EVERY REQUEST

1. Client sends `Authorization: Bearer <accessToken>` .
2. `JwtAuthGuard` triggers Passport, which runs `JwtStrategy` .
3. Passport verifies the signature and expiry using `JWT_SECRET` . If invalid/expired, the request is rejected with `401 Unauthorized` before it ever reaches the controller.
4. If valid, `validate()` returns the payload, which Passport attaches as `request.user` .
5. The controller reads it via `@CurrentUser()` , e.g. `user.sub` is the MongoDB user id.

8. Access Token + Refresh Token Flow

WHAT AND WHY TWO TOKENS

A single long-lived token would be dangerous to keep in browser JavaScript (if stolen via XSS, an attacker has access for a long time). DayFlow instead uses a **short-lived access token** (15 minutes, kept only in a JS variable, never persisted to localStorage) for calling the API, and a **long-lived refresh token** (30 days, stored as an **httpOnly cookie** so client-side JavaScript can never read it) used only to mint new access tokens.

Issuing tokens

WHERE

`AuthService.issueTokens()` , `backend/src/modules/auth/auth.service.ts`

```
private async issueTokens(userId: string, email?: string) {
  const payload = { sub: userId, email };
  const accessToken = await this.jwt.signAsync(payload, {
    secret: JWT_SECRET, expiresIn: JWT_ACCESS_EXPIRES_IN, // default 15m
  });
  const refreshToken = await this.jwt.signAsync(payload, {
    secret: JWT_REFRESH_SECRET, expiresIn: JWT_REFRESH_EXPIRES_IN, // default 30d
  });
  await this.sessions.create({ userId, tokenHash: sha256(refreshToken), expiresAt });
  return { accessToken, refreshToken };
}
```

Note the two tokens are signed with **two different secrets** (`JWT_SECRET` vs `JWT_REFRESH_SECRET`), so a leaked access token can never be used to mint new tokens.

Where each token lives

WHERE

`AuthController` , `backend/src/modules/auth/auth.controller.ts`

```
private setRefreshCookie(res: Response, token: string) {
  res.cookie('refresh_token', token, {
    httpOnly: true,      // JavaScript cannot read this cookie at all
    secure: true,
    sameSite: 'none',    // needed because frontend (9002) and backend (3001) are different origins
    path: '/api/auth',   // only ever sent to /api/auth/* routes, nowhere else
    maxAge: 30 days,
  });
}
```

Every auth endpoint (`register` , `login` , `guest` , `refresh`) calls this and returns only `{ accessToken }` in the JSON body — the refresh token is **never** present in any response body or visible to frontend JavaScript.

The `refresh_sessions` collection (why refresh tokens can be revoked)

WHERE

`backend/src/modules/auth/schemas/refresh-session.schema.ts`

```
@Schema({ timestamps: true, collection: 'refresh_sessions' })
export class RefreshSession {
  userId: Types.ObjectId;
  tokenHash: string;      // SHA-256 hash of the token — the raw token is never stored
  expiresAt: Date;        // TTL index: MongoDB auto-deletes the row after this
  revokedAt?: Date;       // set on logout, refresh, or password reset
}
```

HOW REFRESH + ROTATION WORKS

1. Client's access token expires; the next API call gets **401**.
2. Frontend calls **POST /api/auth/refresh** (no body needed — the httpOnly cookie is sent automatically by the browser).
3. **AuthService.refresh()** verifies the refresh JWT's signature, then looks up its hash in **refresh_sessions**, checking it exists, is not revoked, and has not expired.
4. If valid: that session row is marked **revokedAt** (so it can never be used again — this is **refresh token rotation**), and a brand new access+refresh pair is issued and stored.
5. If invalid: **401 Unauthorized**, and the frontend treats the user as logged out.

How the frontend uses this automatically

WHERE

apiFetch(), **src/lib/api-client.ts**

```
export async function apiFetch<T>(path, init, retry = true) {
  const res = await fetch(url, { ...init, credentials: 'include', headers: { Authorization: `Bearer ${accessToken}` }, ...
  if (res.status === 401 && retry) {
    const refreshed = await refreshSession();      // POST /auth/refresh
    if (refreshed) return apiFetch(path, init, false); // retry the original call ONCE
  }
  if (!res.ok) await parseError(res);
  return res.json();
}
```

This means every page in the app can just call **apiFetch('/tasks')** without ever thinking about token expiry — a mid-session expiry is invisible to the rest of the UI.

Logout and password reset also revoke sessions

logout() revokes the one session tied to the cookie being cleared. **resetPassword()** goes further and revokes **all** of that user's refresh sessions with **sessions.updateMany({userId}, {revokedAt})** — so resetting a password immediately signs the user out everywhere (Section 10 & 16).

9. Password Hashing

WHAT

Passwords are never stored in a form that could be read back. They are hashed with **bcrypt** (via the `bcrypt` npm package), a slow, salted, one-way hashing algorithm purpose-built for passwords.

WHY BCRYPT, AND WHY COST FACTOR 12

Bcrypt is deliberately slow (tunable via a "cost factor") to make brute-force guessing of leaked hashes impractical. Cost factor `12` (used throughout this codebase) is a widely-recommended default that takes roughly 200–300ms per hash on typical hardware — fast enough for a real login, slow enough to make guessing millions of passwords expensive.

Where hashing happens

Operation	File / method	Code
Register	<code>AuthService.register()</code>	<code>passwordHash: await bcrypt.hash(dto.password, 12)</code>
Login	<code>AuthService.login()</code>	<code>await bcrypt.compare(dto.password, user.passwordHash)</code>
Change password	<code>AuthService.changePassword()</code>	compares current password with bcrypt, then re-hashes the new one
Reset password	<code>AuthService.resetPassword()</code>	re-hashes the new password after validating the reset token

The `passwordHash` field is hidden by default

WHERE

`backend/src/modules/users/schemas/user.schema.ts`

```
@Prop({ select: false })
passwordHash?: string;
```

Because of `select: false`, a normal `User.findById(...)` query will **never** return the hash — not even by accident. Anywhere the backend actually needs to compare a password, it explicitly opts in with `.select('+passwordHash')` (see `UsersService.findByEmail` and `findByIdWithPassword`). This means an accidental typo like returning a raw user document from an API response could never leak a password hash.

10. Forgot Password, Password Reset, and Resend

WHAT

A self-service "I forgot my password" flow: the user enters their email, receives an email with a one-time link, and sets a new password from that link — without ever needing their old password.

The moving parts

Piece	File
<code>PasswordResetToken</code> schema	<code>modules/auth/schemas/password-reset-token.schema.ts</code>
<code>MailService</code> (Resend wrapper)	<code>modules/auth/mail.service.ts</code>
<code>RequestPasswordResetDto</code> , <code>ResetPasswordDto</code>	<code>modules/auth/dto/auth.dto.ts</code>
<code>AuthService.requestPasswordReset()</code> / <code>resetPassword()</code>	<code>modules/auth/auth.service.ts</code>
<code>POST /api/auth/forgot-password</code> / <code>reset-password</code>	<code>modules/auth/auth.controller.ts</code> (both public — no <code>JwtAuthGuard</code>)
Login page's "Forgot Password?" dialog	<code>src/app/login/page.tsx</code>
The page the emailed link opens	<code>src/app/reset-password/page.tsx</code>

The token schema

```
@Schema({ timestamps: true, collection: 'password_reset_tokens' })
export class PasswordResetToken {
  userId: Types.ObjectId;
  tokenHash: string; // SHA-256 hash – raw token is never stored
  expiresAt: Date; // TTL index – auto-deleted by MongoDB after 30 minutes
  usedAt?: Date; // set the moment the token is used, enforcing single-use
}
```

Requesting a reset

```
async requestPasswordReset(email: string) {
  const user = await this.users.findByEmail(email);
  if (user?.email) {
    const rawToken = randomBytes(32).toString('hex'); // 256-bit random token
    await this.resetTokens.create({ userId: user._id, tokenHash: sha256(rawToken), expiresAt: now + 30min });
    const resetLink = `${FRONTEND_URL}/reset-password?token=${rawToken}`;
    await this.mail.sendPasswordResetEmail(user.email, resetLink);
  }
  return { success: true }; // ALWAYS success, whether or not the email exists
}
```

WHY IT ALWAYS RETURNS SUCCESS

If the endpoint returned a different response for "email not found" versus "email sent", an attacker could use it to check which email addresses have accounts on DayFlow (email/account enumeration). Returning an identical `{success:true}` either way closes that off. The frontend's copy reflects this too: "If an account exists for that email, we've sent a link..."

Completing a reset

```

async resetPassword(token: string, newPassword: string) {
  const resetToken = await this.resetTokens.findOne({
    tokenHash: sha256(token), usedAt: { $exists: false }, expiresAt: { $gt: new Date() },
  });
  if (!resetToken) throw new UnauthorizedException('Invalid or expired reset link');
  resetToken.usedAt = new Date(); // single-use enforced here
  await resetToken.save();
  await this.users.updatePasswordHash(resetToken.userId, bcrypt.hash(newPassword, 12));
  await this.sessions.updateMany({ userId: resetToken.userId }, { revokedAt: new Date() }); // sign out everywhere
}

```

Sending the email — Resend

WHAT IS RESEND

Resend is a transactional email API. `mail.service.ts` wraps it in one method, `sendPasswordResetEmail(to, resetLink)`.

```

constructor(private readonly config: ConfigService) {
  const apiKey = config.get('RESEND_API_KEY');
  this.fromAddress = config.get('MAIL_FROM_ADDRESS', 'DayFlow <onboarding@resend.dev>');
  this.client = apiKey ? new Resend(apiKey) : null;
  if (!this.client) console.warn('RESEND_API_KEY is not configured. Password reset emails will not be sent.');
```

GRACEFUL DEGRADATION

If `RESEND_API_KEY` is not set, the app still works end-to-end — it just silently skips the actual send (with a console warning) instead of crashing. This matches the same pattern used for VAPID/push keys (Section 15).

IMPORTANT OPERATIONAL LIMITATION

Resend accounts without a verified sending domain run in "sandbox mode": the `from` address must be Resend's own `onboarding@resend.dev` (you cannot send *from* a domain you don't own, like a personal Gmail address), and email can currently only be delivered to the developer's own verified Resend account address, not to arbitrary users. This was discovered and fixed during testing — see Section 17. Real end users will not be able to receive reset emails until a custom domain is verified on the Resend account.

11. User Authentication and Ownership / Authorization

AUTHENTICATION VS. AUTHORIZATION

Authentication answers "who is making this request?" — handled entirely by `JwtAuthGuard` + `JwtStrategy` (Section 7).

Authorization answers "is this user allowed to touch this specific piece of data?" — handled by every service method filtering its MongoDB query by that user's id.

The rule, applied everywhere

Every single feature query in the backend takes exactly this shape:

```
this.model.find({ userId })           // list
this.model.create({ ...dto, userId }) // create
this.model.findOneAndUpdate({ _id: id, userId }, dto) // update
this.model.deleteOne({ _id: id, userId }) // delete
```

`userId` here always comes from `@CurrentUser()` → `user.sub` → the verified JWT payload. **It is never taken from the request body, a URL parameter, or any other client-controlled input.** This is what makes it impossible for User A to read, edit, or delete User B's data, even if they know the exact MongoDB `_id` of User B's document: the

`findOneAndUpdate` / `deleteOne` filters on *both* `_id` and `userId`, so a mismatched owner simply matches zero documents.

Guest accounts

`POST /api/auth/guest` creates a real MongoDB user document (`{username: 'Guest User', isAnonymous: true}` , no email, no password) and issues a normal JWT for it. From the rest of the backend's point of view, a guest is authenticated exactly like any other user — the only special-casing is `isAnonymous` being used by the frontend to hide account-security UI (email/password change) that doesn't apply to a passwordless account.

Anonymous data isolation via `sparse` email index

```
@Prop({ unique: true, lowercase: true, trim: true, sparse: true })
email?: string;
```

`sparse: true` means the unique index only applies to documents that *have* an `email` field at all — so MongoDB allows unlimited guest users with no email, while still enforcing that every registered (non-guest) email is unique.

12. All API Endpoints, Grouped by Module

All routes are prefixed with `/api` (set globally in `main.ts` via `app.setGlobalPrefix('api')`). "Auth" column shows whether `JwtAuthGuard` is required.

Auth — /api/auth

Endpoint	Auth	Purpose
POST /register	Public	Create an account (email + password + username)
POST /login	Public	Sign in with email + password
POST /guest	Public	Create a passwordless guest account
POST /refresh	Cookie	Exchange the refresh cookie for a new access token (rotates the session)
POST /logout	Cookie	Revoke the current refresh session
GET /me	JWT	Return the decoded JWT payload (sub, email)
PATCH /change-email	JWT	Change email (requires current password)
PATCH /change-password	JWT	Change password (requires current password)
POST /forgot-password	Public	Send a password-reset email if the address is registered
POST /reset-password	Public (token in body)	Set a new password using a valid, unused, unexpired reset token

Users — /api/users (all JWT)

Endpoint	Purpose
GET /me	Fetch the current user's public profile
PATCH /me	Update username, email, phoneNumber, photoUrl, or theme

Tasks — /api/tasks (all JWT)

Endpoint	Purpose
GET /	List the user's tasks, newest first
POST /	Create a task
PATCH /:id	Update a task (full <code>TaskDto</code> required, incl. <code>text</code>)
DELETE /:id	Delete a task

Expenses — /api/expenses (all JWT)

Endpoint	Purpose
GET /transactions	List transactions, newest date first
POST /transactions	Create a transaction
PATCH /transactions/:id	Update a transaction (partial)
DELETE /transactions/:id	Delete a transaction
GET /budget	Get the monthly budget (auto-created with a default of 5,000,000 cents on first read)
PATCH /budget	Set the monthly budget

Habits — /api/habits (all JWT)

Endpoint	Purpose
GET /	List habits
PATCH / REPLACE-ALL	Replace the <i>entire</i> habit list (deletes all, re-inserts the given array)
GET /gym	Get the Gym Tracker data blob (workout plan, nutrition logs, feature toggle)
PATCH /gym REPLACE-ALL	Replace the entire Gym Tracker data blob

Planner — /api/planner (all JWT)

Endpoint	Purpose
GET /	Get the weekly schedule (auto-created empty on first read)
PUT / REPLACE-ALL	Replace the entire <code>days</code> map

Notes — /api/notes (all JWT)

Endpoint	Purpose
GET /	List notes, newest first
POST /	Create a note (text or checklist)
PATCH /:id	Update a note (partial)
DELETE /:id	Delete a note

Credentials (Password Manager) — /api/credentials (all JWT)

Endpoint	Purpose
GET /	List saved credentials
POST /	Create a credential entry
PATCH /:id	Update a credential entry (partial)
DELETE /:id	Delete a credential entry

Notifications — /api/notifications (all JWT)

Endpoint	Purpose
GET /	List notifications, newest first
POST /	Create a notification/reminder
PATCH /:id	Update a notification (e.g. mark read)
DELETE /:id	Delete a notification

Push — /api/push (all JWT)

Endpoint	Purpose
POST /subscriptions	Save (upsert by endpoint) a browser push subscription
DELETE /subscriptions	Remove a push subscription
POST /test	Placeholder — returns success without sending (real delivery lives in JobsModule)

Jobs — /api/jobs (Cron-secret protected, not JWT)

Endpoint	Purpose
POST /send-reminders	Push today's due in-app reminders to subscribed devices
POST /water-check	Push a hydration nudge to users behind their water-intake goal at 11am/5pm/9pm checkpoints

Both require header `Authorization: Bearer <CRON_SECRET>` instead of a user JWT — they are meant to be triggered by an external scheduler, not a logged-in user.

Health — /api/health

Endpoint	Purpose
GET /database	Pings MongoDB; returns 200 if connected, 503 if not

13. Frontend → api-client → NestJS → MongoDB Request Flow

WHERE

`src/lib/api-client.ts` is the single file every page/hook goes through to talk to the backend.

The building blocks of `api-client.ts`

Export	Role
<code>API_BASE_URL</code>	Read from <code>NEXT_PUBLIC_BACKEND_URL</code> (defaults to <code>http://localhost:3001/api</code>)
module-level <code>accessToken</code> variable	Held only in memory (not localStorage) — cleared on full page reload, restored via <code>refreshSession()</code>
<code>login/register/guestLogin</code>	Call the token-issuing auth endpoints, store the returned access token
<code>refreshSession()</code>	Calls <code>/auth/refresh</code> ; used on app load and by <code>apiFetch</code> 's auto-retry
<code>apiFetch<T>(path, init)</code>	The generic, authenticated fetch wrapper used by every feature page
<code>ApiError</code>	A custom <code>Error</code> subclass carrying the HTTP status code, so callers can branch on e.g. <code>error.status === 401</code>

A typical call, start to finish

```
// in a page component, e.g. src/app/dashboard/page.tsx
const created = await apiFetch<ApiTask>('/tasks', {
  method: 'POST',
  body: JSON.stringify({ text, completed: false, priority }),
});
```

1. **apiFetch** builds the full URL (`API_BASE_URL + '/tasks'`), attaches `Content-Type: application/json` (because a body is present), `Authorization: Bearer <accessToken>` , and `credentials: 'include'` (so the httpOnly refresh cookie travels with the request even though this call doesn't need it directly).
2. The browser sends the request cross-origin (frontend on :9002, backend on :3001).
3. **NestJS's CORS middleware** (configured in `main.ts` with `origin: FRONTEND_URL, credentials: true`) allows it.
4. **Global body parser, cookie parser, ValidationPipe** run (Section 14).
5. **JwtAuthGuard** verifies the token; **TasksController.create** runs; **TasksService.create** inserts into MongoDB.
6. The created document (with its new `_id` , `createdAt` , `updatedAt`) comes back as JSON.
7. **apiFetch** checks `res.ok` ; if not ok, throws `ApiError` . If ok, returns `res.json()` to the caller.
8. The page maps the raw `{_id, ...}` shape into the frontend's own `TodoItem` type (`_id` → `id`) and updates React state.

See Section 20 for the fully worked version of this exact example, including the 401/refresh path.

14. Validation and Error Handling

Request validation — the global `ValidationPipe`

WHERE

`backend/src/main.ts`

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,           // strip any property not declared on the DTO
  transform: true,           // convert plain JSON into an instance of the DTO class
  forbidNonWhitelisted: true, // REJECT (400) requests that include undeclared properties
}));
```

WHY `FORBIDNONWHITELISTED` MATTERS

This makes every DTO an explicit allow-list. If a client sends a field the DTO doesn't declare (e.g. trying to sneak a `userId` into a task-creation body), the whole request is rejected with `400 Bad Request` instead of the extra field being silently ignored or, worse, accidentally used.

class-validator decorators used across DTOs

Decorator	Meaning
<code>@IsString()</code> , <code>@IsBoolean()</code> , <code>@IsInt()</code> , <code>@IsObject()</code>	Type check
<code>@IsEmail()</code>	Must look like a valid email address
<code>@IsOptional()</code>	Field may be omitted entirely
<code>@IsDefined()</code>	Field must be present, even if its type is <code>unknown</code> (used for <code>Note.content</code> , see Section 17)
<code>@MinLength()</code> / <code>@MaxLength()</code>	String length bounds
<code>@Min()</code>	Numeric lower bound (e.g. amounts ≥ 0)
<code>@IsIn([...])</code>	Value must be one of a fixed set (e.g. task priority, credential category)
<code>@Matches(regex)</code>	Must match a pattern — used to enforce <code>YYYY-MM-DD</code> date strings

Uniform error responses

WHERE

`backend/src/common/filters/http-exception.filter.ts`

```
@Catch()
export class HttpExceptionHandler implements ExceptionFilter {
  catch(exception, host) {
    const status = exception instanceof HttpException ? exception.getStatus() : 500;
    const message = ... // extracted from the exception, or 'Internal server error'
    response.status(status).json({ statusCode: status, message, path: request.url, timestamp });
  }
}
```

This is registered globally in `main.ts` with `app.useGlobalFilters(...)`, so every error — a thrown `UnauthorizedException`, a validation failure, an unexpected exception — comes back to the frontend in the exact same JSON shape.

How the frontend consumes errors

```
// src/lib/api-client.ts
async function parseError(res: Response): Promise<never> {
  const body = await res.json();
  const message = Array.isArray(body.message) ? body.message.join(', ') : body.message;
  throw new ApiError(res.status, message);
}
```

Pages then branch on the status code, e.g. in `login/page.tsx`: `if (error instanceof ApiError && error.status === 401) { ... }` to show "Invalid credentials" specifically for a failed login, versus a generic message for anything else.

15. Notifications, Push, and Background Jobs

Three related but distinct concepts

Concept	Module	What it is
In-app notifications	<code>NotificationsModule</code>	Plain reminder records (title, date, message, read flag) shown in the notification bell / notifications page
Push subscriptions	<code>PushModule</code>	The Web Push API subscription object a browser gives you when a user allows notifications — stored so the server can later push to that specific browser/device
Background jobs	<code>JobsModule</code>	Endpoints meant to be triggered by an external scheduler (cron), which scan the database and actually send push notifications via the <code>web-push</code> library

How a push notification actually gets delivered

HOW

1. Browser calls `registration.pushManager.subscribe(...)` and gets back an `endpoint` + encryption `keys`.
2. Frontend sends that to `POST /api/push/subscriptions`, stored in the `push_subscriptions` collection (one document per browser/device, upserted by `endpoint`).
3. An external scheduler calls `POST /api/jobs/send-reminders` or `/water-check` with the `CRON_SECRET`.
4. `JobsService` queries MongoDB directly (today's notifications, or habits with a water-tracking target), and for each match calls `webpush.sendNotification({endpoint, keys}, payload)` for every subscription belonging to that user.
5. If a push fails with HTTP 404/410 (subscription no longer valid — e.g. the user uninstalled/blocked notifications), that subscription document is deleted automatically.

VAPID keys

WHAT

VAPID (Voluntary Application Server Identification) keys are a public/private key pair that identifies your server to push services (Chrome, Firefox, etc.) when sending Web Push notifications.

```
// backend/src/modules/jobs/jobs.module.ts
private ensureVapid() {
  const publicKey = config.get('VAPID_PUBLIC_KEY');
  const privateKey = config.get('VAPID_PRIVATE_KEY');
  const subject = config.get('VAPID_SUBJECT');
  if (!publicKey || !privateKey || !subject) {
    console.warn('VAPID keys are not fully configured. Push notifications will not be sent.');
```

CURRENT STATUS

VAPID keys are **not set** in this project's environment (this was true even before the Firebase migration). Every job endpoint still runs successfully and returns `{success:true, message:'VAPID keys not configured...'}` — it just performs no actual sends. This is a pre-existing gap, not a bug introduced by the migration.

Why the cron logic lives in the backend, not Next.js, now

Before the migration, `src/app/api/cron/send-reminders/route.ts` and `water-check/route.ts` used the Firebase Admin SDK to loop over every Firestore user document. Those Next.js API routes were deleted, and the same logic (query → filter → push) was reimplemented in `JobsModule` against MongoDB — and simplified, since MongoDB lets you query "all notifications dated today across all users" in a single query, instead of looping over every user document and then a per-user sub-collection the way Firestore's nested structure required.

16. Important Security Decisions

Decision	Reasoning
Refresh token in an httpOnly cookie, never in a JSON response or localStorage	Client-side JavaScript (and therefore XSS) can never read it
Access token kept only in memory, not persisted	Limits the exposure window if it were ever exfiltrated; a reload requires a fresh <code>/auth/refresh</code>
Only <i>hashes</i> of refresh tokens and password-reset tokens are stored (SHA-256)	A database read/leak alone can't be used to forge a session or reset link — the raw value is required and only ever exists in the one-time email/response
Password-reset tokens are single-use (<code>usedAt</code>) and short-lived (30 minutes, MongoDB TTL index)	Shrinks the window an intercepted reset link could be replayed
Resetting a password revokes <i>all</i> of that user's refresh sessions	If the password was compromised, any existing logged-in session (attacker's or otherwise) is also cut off
Forgot-password always responds <code>{success:true}</code>	Prevents using the endpoint to enumerate which emails have accounts
Passwords hashed with bcrypt, cost factor 12	Industry-standard slow hash, resistant to brute force even if the database leaks
<code>passwordHash</code> is <code>select: false</code> by default	Prevents accidental inclusion of the hash in any API response
Every DB query is filtered by the JWT's <code>sub</code> (userId), never a client-supplied id	The core of the authorization model — see Section 11
Global <code>ValidationPipe</code> with <code>forbidNonWhitelisted:true</code>	Rejects requests carrying fields the DTO didn't declare, closing off a whole class of injection/override attempts
CORS restricted to a single configured origin (<code>FRONTEND_URL</code>), not <code>*</code>	Required for cookies to work cross-origin at all, and prevents arbitrary sites from calling the API with a user's credentials
Refresh cookie scoped to <code>path: '/api/auth'</code>	The browser will not attach it to unrelated requests (e.g. <code>/api/tasks</code>), reducing exposure

17. Important Bugs Discovered and Fixed During Migration

These are real issues hit and fixed while building this backend and migrating the frontend to it — not hypothetical examples.

1. Backend wouldn't start — missing dependency

`@nestjs/platform-express` (the HTTP driver NestJS needs) was not installed. Fixed by installing the version matching the other `@nestjs/*` packages (v11).

2. Stale incremental build cache masked a missing build

`tsconfig.tsbuildinfo` persisted after `dist/` was deleted, causing TypeScript to believe the output was already up to date and skip re-emitting files entirely. Fixed by deleting the build-info file to force a clean rebuild.

3. Malformed `MONGODB_URI`

The connection string in `.env` was, at different points, just a bare password fragment, and later a value with the `MONGODB_URI=` key itself accidentally missing from the line (a copy/paste artifact) — both silently produced a connection failure rather than an obviously-labelled error.

4. Root `tsconfig.json` was compiling the backend by accident

The frontend's TypeScript config had no `exclude` entry for `backend/`, so running `tsc` at the repo root tried to type-check the NestJS backend using the *frontend's* compiler options (no decorator support), producing hundreds of unrelated errors. Fixed by adding `"backend"` to the root `exclude` array.

5. Signup appeared to fail, but the account was actually created

`use-auth.tsx` tried to bootstrap a Firestore profile document for every newly authenticated user — including users authenticated through the new NestJS/JWT flow, who have no Firebase Auth session at all. Firestore's security rules require `request.auth != null`, so every one of those writes failed with `permission-denied`, and that thrown error bubbled up and was shown as "Sign-Up Failed" even though the MongoDB user was created successfully. Fixed by removing the Firestore bootstrap from the backend-authenticated code path.

6. Notes could never be created — a missing validator

`NoteDto.content` had no `class-validator` decorator at all. Combined with the global `ValidationPipe`'s `forbidNonWhitelisted: true`, this meant the `content` field was treated as undeclared and every `POST / PATCH /notes` was rejected with `400 property content should not exist`. Found while migrating the Notes page; fixed with a single `@IsDefined()` decorator.

7. Destructive "replace-all" semantics on Habits/Planner, discovered the hard way

`PATCH /api/habits` and `PUT /api/planner` do not merge — they delete everything and reinsert exactly what was sent. Early testing that sent a partial list silently deleted the rest of a user's habits. Documented and guarded against by always sending the *full* current array/object from the frontend (see the code comments in `habits/page.tsx`).

8. Concurrent builds corrupted the shared backend

While multiple parts of the migration were being worked on in parallel, more than one process ran `nest build` at the same time. Because Nest's build wipes `dist/` before regenerating it (`deleteOutDir: true`), the concurrent wipes and writes corrupted the output and crashed the running server (`Cannot find module './modules/auth/auth.module'`). Fixed by serializing backend rebuild/restart to a single owner instead of letting it happen from multiple places at once.

9. A stale Firebase session bypassed the new login entirely

Firebase persists its auth session in the browser (IndexedDB) across reloads. A browser that had an old Firebase session (from earlier Guest-mode testing) would auto-restore that session and skip the new backend's login screen — but that session carried no NestJS JWT, so every save silently failed with `401`. It looked like "the app just isn't saving anything." Diagnosed by checking which auth path was actually active; resolved by logging out (or clearing site data) to force a real login through the new backend.

10. Resend rejected the email — twice, for two different reasons

First attempt: sending to a `@example.com` test address was rejected (`422`) — Resend's sandbox mode only allows sending to the account's own verified address. Second attempt (correct recipient): sending *from* `pranathisundara3@gmail.com` was rejected (`403 domain not verified`) — Resend requires the *from* domain to be owned and verified, and nobody can verify `gmail.com`. Fixed by setting `MAIL_FROM_ADDRESS` back to Resend's sandbox sender, `onboarding@resend.dev`.

11. `PASSWORD_ENCRYPTION_KEY` is defined but never implemented

Discovered while migrating the Password Manager feature: despite the field being named `encryptedData` and an encryption key existing in `.env.example`, no encryption code exists anywhere in the codebase (confirmed by searching the entire backend source for the variable — it is not referenced). Credential values are stored as plain strings in MongoDB; the UI only masks them visually with dots. This was **not** introduced by the migration — the same was true of the original Firestore implementation — but it is a real, currently-unaddressed gap (see Section 22).

18. Environment Variables

NOTE

No real secret values are included below — only variable names and what they are for.

Backend — backend/.env

Variable	Used for
<code>NODE_ENV</code>	Standard Node environment flag
<code>PORT</code>	Port the NestJS server listens on (3001)
<code>MONGODB_URI</code>	MongoDB Atlas connection string
<code>MONGODB_SERVER_SELECTION_TIMEOUT_MS</code>	How long Mongoose waits to find a usable MongoDB server before failing
<code>MONGODB_CONNECT_TIMEOUT_MS</code>	How long Mongoose waits for the initial connection
<code>FRONTEND_URL</code>	Used for the CORS allow-list <i>and</i> to build password-reset links
<code>JWT_SECRET</code>	Signs/verifies access tokens
<code>JWT_REFRESH_SECRET</code>	Signs/verifies refresh tokens (deliberately different from <code>JWT_SECRET</code>)
<code>JWT_ACCESS_EXPIRES_IN</code>	Access token lifetime (default <code>15m</code>)
<code>JWT_REFRESH_EXPIRES_IN</code>	Refresh token lifetime (default <code>30d</code>)
<code>CRON_SECRET</code>	Bearer secret required to call the <code>/api/jobs/*</code> endpoints
<code>PASSWORD_ENCRYPTION_KEY</code>	Defined but not used anywhere in the current code (see Section 17, bug 11)
<code>RESEND_API_KEY</code>	Authenticates with the Resend email API for password-reset emails
<code>MAIL_FROM_ADDRESS</code>	The "from" address/name used on password-reset emails
<code>VAPID_PUBLIC_KEY</code> / <code>VAPID_PRIVATE_KEY</code> / <code>VAPID_SUBJECT</code>	Read by <code>JobsModule</code> for Web Push delivery. Not present in <code>.env.example</code> — a documentation gap worth fixing, though the code degrades gracefully when they're absent (Section 15)

Frontend — .env.local (project root)

Variable	Used for
<code>NEXT_PUBLIC_BACKEND_URL</code>	Base URL the frontend's <code>api-client.ts</code> calls (e.g. <code>http://localhost:3001/api</code>)
<code>NEXT_PUBLIC_VAPID_PUBLIC_KEY</code>	Read in <code>settings/page.tsx</code> when subscribing the browser to Web Push. Not currently set locally — push subscribe will show a "not configured" error until it is.

19. How to Run Frontend and Backend Locally

Prerequisites

- Node.js and npm
- A MongoDB Atlas connection string (or any reachable MongoDB instance)
- Backend `.env` filled in based on `backend/.env.example` (Section 18)

Backend

```
cd backend
npm install
npm run build      # compiles TypeScript to dist/
npm run start      # node dist/main.js – listens on http://localhost:3001
```

Verify it's up: `curl http://localhost:3001/api/health/database` should return `{"status": "ok", ...}`.

A CAVEAT FOUND DURING DEVELOPMENT

`npm run start:dev` (`nest start --watch`) was occasionally flaky on this specific machine/folder — the watcher sometimes tried to launch `dist/main.js` before the compiler finished writing it, throwing `Cannot find module 'dist/main'`. If that happens: stop the process, delete `tsconfig.tsbuildinfo` and `dist/`, then re-run `npm run build && npm run start`.

Frontend

```
npm install
npx next dev --port 9002 --webpack
```

WHY NOT JUST NPM RUN DEV

This project is on Next.js 16, which defaults to the Turbopack bundler, but the project also has a webpack-based PWA plugin (`@ducanh2912/next-pwa`) configured in `next.config.mjs`. Running plain `next dev` fails outright because of that conflict. The `--webpack` flag forces the same bundler the production `build` script already uses.

Once running, open `http://localhost:9002`.

20. End-to-End Example: Creating a Task

This traces one real user action — adding a to-do item from the Dashboard — through every layer of the system.

Step 1 — User action

On `src/app/dashboard/page.tsx`, the user types a task into the "Add a new task" dialog and clicks **Add Task**, which calls `addTodo()`.

Step 2 — Frontend calls the API client

```
const created = await apiFetch<ApiTask>('/tasks', {
  method: 'POST',
  body: JSON.stringify({ text: newTodoText.trim(), completed: false, priority: newTodoPriority }),
});
```

Step 3 — api-client.ts builds the real HTTP request

```
POST http://localhost:3001/api/tasks
Content-Type: application/json
Authorization: Bearer eyJhbGciOi... (the in-memory access token)
Cookie: refresh_token=... (httpOnly, sent automatically by the browser)

{"text":"Finish Q3 report","completed":false,"priority":"low"}
```

Step 4 — NestJS receives the request

1. **CORS check** (`main.ts`) — origin must match `FRONTEND_URL`.
2. **Body/cookie parsing** (`json()`, `cookieParser()` middleware).
3. **Global ValidationPipe** validates the body against `TaskDto`: `text` is a non-empty string ≤ 500 chars (present, OK), `completed` is a boolean (present, OK), `priority` is one of `high/medium/low` (present, OK). If anything failed here, the request would stop right now with `400 Bad Request` and never reach the controller.
4. **JwtAuthGuard** runs `JwtStrategy.validate()`: verifies the access token's signature against `JWT_SECRET` and checks it hasn't expired. If it's expired or invalid $\rightarrow$ `401` immediately.
5. **TasksController.create()** runs, with `@CurrentUser()` giving it `{sub: '6aa...', email: '...'}` and `@Body()` giving it the validated `TaskDto` instance.

Step 5 — Service writes to MongoDB

```
create(userId: string, dto: TaskDto) {
  return this.model.create({ ...dto, userId });
}
```

Mongoose inserts a new document into the `tasks` collection: `{_id: ObjectId(...), userId: ObjectId('6aa...'), text: 'Finish Q3 report', completed: false, priority: 'low', createdAt: ..., updatedAt: ..., __v: 0}`.

Step 6 — Response

NestJS serializes the created Mongoose document to JSON and returns it with status `201 Created` (Nest's default for a `POST` handler with no explicit `@HttpCode`).

Step 7 — Frontend updates the UI

```
const mapApiTask = (t: ApiTask): TodoItem => ({ id: t._id, text: t.text, completed: t.completed, priority: t.priority });
setTodos([mapApiTask(created), ...todos]); // new task appears at the top of the list
```

What happens if the access token had expired at Step 4?

HOW THE RETRY PATH WORKS

1. Backend returns `401` before ever running `TasksController`.
2. `apiFetch` sees `res.status === 401 && retry`, and calls `refreshSession()`, which `POST`s to `/api/auth/refresh` (the `httpOnly` cookie goes along automatically).
3. `AuthService.refresh()` verifies the refresh JWT, finds and revokes the matching row in `refresh_sessions`, and issues a brand-new access+refresh pair (rotation).
4. `apiFetch` retries the *original* `POST /tasks` request exactly once, now with the new access token, and the flow above continues normally.
5. If the refresh itself fails (refresh token also expired/revoked), the user is effectively logged out and would need to sign in again.

21. What Was Removed From Firebase, and Why

Files deleted

File / directory	Reason
<code>src/lib/firebase.ts</code>	Client SDK initialization (Auth, Firestore, Storage) — no longer needed once every feature moved to the NestJS API
<code>src/lib/firebase-admin.ts</code>	Server-side Admin SDK, used only by the deleted Next.js API routes below
<code>src/lib/web-push.ts</code>	VAPID/web-push initializer used only by the deleted cron routes; the same logic now lives inside <code>JobsModule</code>
<code>src/app/api/push/{subscribe,unsubscribe,send-test}/route.ts</code>	Verified Firebase ID tokens and proxied to Firestore; replaced by the frontend calling the NestJS <code>PushModule</code> endpoints directly
<code>src/app/api/cron/{send-reminders,water-check}/route.ts</code>	Used Firebase Admin to scan Firestore; logic ported into <code>JobsModule</code> against MongoDB
<code>src/app/login/actions.ts</code>	Dead file, already unused before this migration (contained only a comment)
<code>apphosting.yaml</code>	Firebase App Hosting deployment config — no longer applicable
<code>.idx/dev.nix</code>	Firebase Studio (Project IDX) workspace config that auto-started Firebase Auth/Firestore emulators

npm packages removed

`firebase`, `firebase-admin` — removed from the frontend's `package.json`.

What deliberately stayed, and why

Nothing Firebase-related remains in the codebase. Two things that might look similar are not Firebase: profile photos are stored as base64 `data:` URIs directly in MongoDB (no external file storage was introduced), and push notifications use the vendor-neutral Web Push standard (`web-push` npm package, VAPID keys) rather than Firebase Cloud Messaging.

The overall motivation

- **Full data ownership** — all user data lives in a MongoDB database the project controls directly, not a third-party vendor's proprietary database.
- **One consistent backend** instead of a split between client-side Firestore access, Firebase security rules, and a handful of server-side Next.js API routes using Firebase Admin.
- **Avoiding the exact class of bugs Firebase's split model caused** during this project — e.g. bug #5 in Section 17, where a user could be validly authenticated by the new backend but still get rejected by Firestore's separate security-rule engine, which only trusts Firebase Auth sessions.

22. Important Limitations That Still Exist

Limitation	Detail
Password Manager has no real encryption	Credential values are stored as plain strings in MongoDB (<code>encryptedData: Record<string,string></code> is just a bag of unencrypted fields). <code>PASSWORD_ENCRYPTION_KEY</code> exists in <code>.env.example</code> but is not implemented anywhere. The UI only visually masks values with dots.
Push notifications require manual VAPID setup	Neither the backend nor frontend <code>.env</code> currently has real VAPID keys; the "Send Test Notification" button and the reminder/water-check jobs will report success but deliver nothing until keys are generated and set.
Password-reset email delivery is sandboxed	Without a verified sending domain on Resend, real end users (anyone other than the developer's own verified Resend account email) cannot currently receive password-reset emails.
Destructive replace-all endpoints are easy to misuse	<code>PATCH /habits</code> and <code>PUT /planner</code> silently delete anything not included in the request. There is no partial-update variant, so any future client code must remember to always send the complete array/object.
No account recovery beyond password reset	No 2FA, no backup codes, no "forgot username" flow, no email-change confirmation step (change-email takes effect immediately once the current password is verified).
Guest accounts cannot be "upgraded"	There is no flow to convert a guest (passwordless) account into a real email/password account — if the session/cookie is lost, that guest's data becomes unreachable.
No object/file storage	Profile photos are capped at ~1MB and stored inline as base64 inside the user document, which is simple but does not scale well and bloats document size/backup size.
No rate limiting	<code>login</code> , <code>register</code> , and <code>forgot-password</code> have no throttling, making them susceptible to brute-force or spam without an added rate-limiting layer.
No automated test suite	All verification during the migration was done manually via <code>curl</code> scripts against a running backend, not unit or integration tests committed to the repo.
VAPID env vars undocumented	<code>VAPID_PUBLIC_KEY</code> / <code>VAPID_PRIVATE_KEY</code> / <code>VAPID_SUBJECT</code> are read by <code>JobsModule</code> but not listed in <code>backend/.env.example</code> .

— End of document —